Asynchronous programming changes the way that programs are structured. Further, not only are all programs in CoffeeScript asynchronous, but they also run on a single event loop. This presents unique challenges when you're writing programs.

In this chapter you'll start by processing some data synchronously and see how far you can get before a synchronous approach breaks down. Then you'll see how to write programs using an asynchronous approach and learn how to manage the challenges that asynchronous programming presents.

9.1. Data processing

All programs need to deal with data: reading data in, processing (usually), and writing data out. Although the most common way to read, process, and write is synchronously, the synchronous approach quickly breaks down in a CoffeeScript program due to the nature of the event loop. In this section you'll find out exactly why it breaks down and what you can do about it.

Imagine Agtron and Scruffy are organizing the World Volcano Mogul Skiing Championship. You're working with them to display the names of competitors, one per second, on electronic billboards placed strategically across the volcano tops. The names must be displayed in alphabetical order, but they're supplied in random order in a file called competitors with each line containing one competitor in the format *Competitor number: Last name, First name*:

```
0212: Turnbill, Geralyn
0055: Spielvogel, Cierra
0072: Renyer, Connie
0011: Engholm, Ciara
0088: Gitting, Estrella
```

The entire file contains 1,500 names. To sort the names, you'll first need to read the file into an array. Suppose you're using Node.js to do this, and that, without knowing any better, you're doing it synchronously. What happens?

9.1.1. Reading